

```
In [... import numpy as np
import matplotlib.pyplot as plt
import timeit
```

NumPy

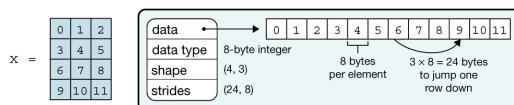
- Efficient and powerful library of arrays
- NumPy arrays are more efficient than Python lists as they are stored in a contiguous block of memory.
- Many functions are available for mathematical operations, linear algebra, and so on.

Resources:

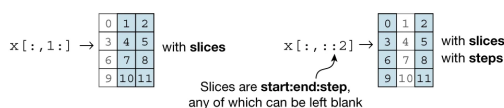
- Beginner: [Numpy official tutorial](#)
- [Official Numpy Quickstart](#)
- [A visual guide to Numpy](#)
- Somewhat advanced: [Nicolas P. Rougier's tutorial](#)
- [M Scott Shell's notes](#)
- Harris, C. R. et al. Array Programming with NumPy. Nature 2020, 585 (7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>.

A neat summary of core numpy methods

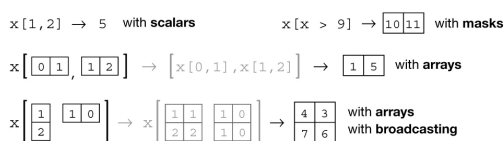
a Data structure



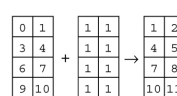
b Indexing (view)



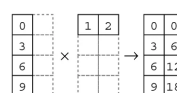
c Indexing (copy)



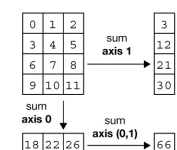
d Vectorization



e Broadcasting



f Reduction



g Example

```
In [1]: import numpy as np
In [2]: x = np.arange(12)
In [3]: x = x.reshape(4, 3)

In [4]: x
Out[4]:
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])

In [5]: np.mean(x, axis=0)
Out[5]: array([4.5, 5.5, 6.5])

In [6]: x = x - np.mean(x, axis=0)

In [7]: x
Out[7]:
array([[ -4.5, -4.5, -4.5],
       [ -1.5, -1.5, -1.5],
       [  1.5,  1.5,  1.5],
       [  4.5,  4.5,  4.5]])
```

Source: Harris et al., Nature (2020). See above

Defining arrays

```
In [... # 1D python list
foo = [1, 2, 3, 4, 5]

# a use case: finding mean of a list of numbers
# using only python
sum=0
for i in foo:
    sum = sum+i
print(sum/5)

# using numpy
arr1 = np.array(foo)
# print(arr1,arr1[2])
print(f'Mean = {arr1.mean()}, sum = {arr1.sum()}')

3.0
Mean = 3.0, sum = 15
```

```
In [... # 2D array
arr2 = np.array([[1, 2, 3], [4, 5, 6]])

print(arr2)
print(arr2[1,2])

[[1 2 3]
 [4 5 6]]
6
```

Array attributes

```
In [... print(arr2)
print(arr2.shape) #shape of the array
print(arr2.dtype) # data type
print(arr2.size) # size of the array
print(arr2.ndim) # number of dimensions

[[1 2 3]
 [4 5 6]]
(2, 3)
int64
6
2
```

```
In [... # 2d arrays in python
a = []
for i in range(2):
    b = []
    for j in range(2):
        b.append(j)
    a.append(b)
print(a)

# numpy
a = np.tile(np.arange(2), (2, 1))
print(a)

[[0, 1], [0, 1]]
[[0 1]
 [0 1]]
```

Initializing arrays

```
In [... zeros_arr = np.zeros((4, 4))
zeros_arr
```

```
Out[... array([[0., 0., 0., 0.],
              [0., 0., 0., 0.],
              [0., 0., 0., 0.],
              [0., 0., 0., 0.]])
```

```
In [... ones_arr = np.ones((2, 4))
ones_arr
```

```
Out[... array([[1., 1., 1., 1.],
              [1., 1., 1., 1.]])
```

```
In [... full_arr = np.full((3,2), 'adsfljd')
full_arr
```

```
Out[... array([[ 'adsfljd', 'adsfljd'],
              [ 'adsfljd', 'adsfljd'],
              [ 'adsfljd', 'adsfljd']], dtype='<U7')
```

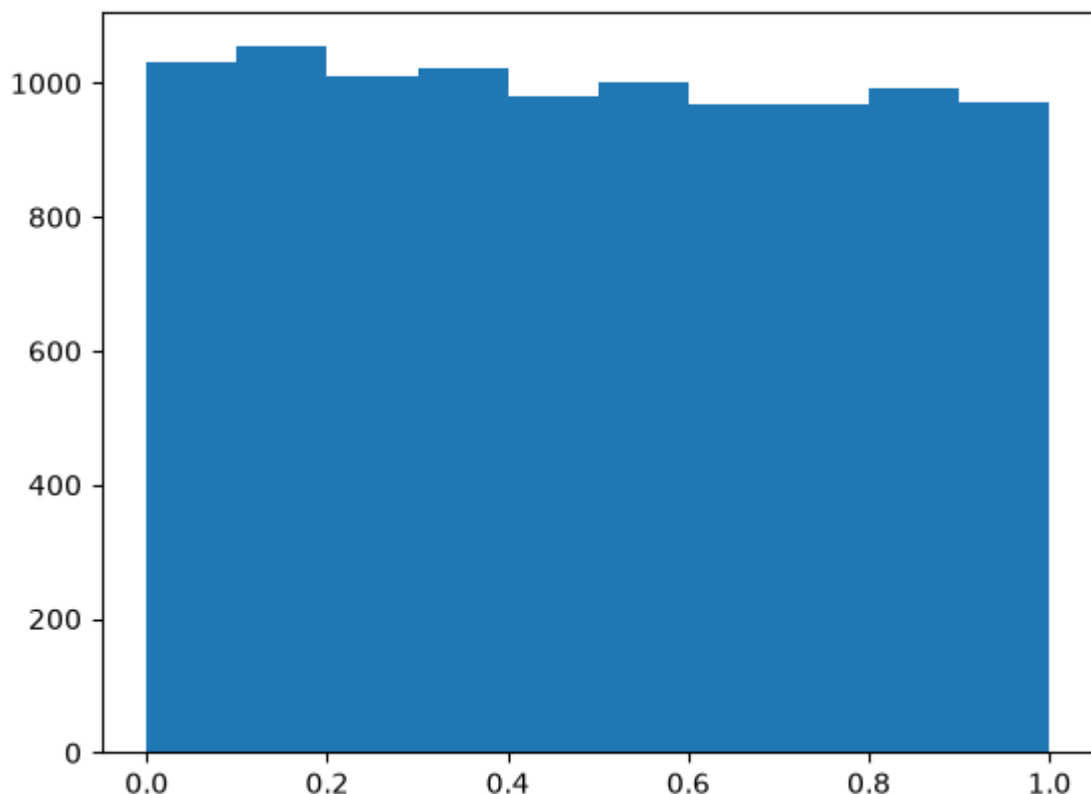
```
In [... identity_matrix = np.eye(3)
identity_matrix
```

```
Out[... array([[1., 0., 0.],
              [0., 1., 0.],
              [0., 0., 1.]])
```

```
In [... # Array with random values
# np.random.seed(4)
random_arr = np.random.random((10000))

plt.hist(random_arr)
random_arr
```

```
Out[... (array([1031., 1054., 1009., 1023., 981., 1002., 9
69., 969., 992.,
970.]),
array([6.36604711e-05, 1.00049200e-01, 2.00034739
e-01, 3.00020278e-01,
4.00005817e-01, 4.99991356e-01, 5.99976895
e-01, 6.99962434e-01,
7.99947973e-01, 8.99933512e-01, 9.99919052
e-01])),
<BarContainer object of 10 artists>)
Out[... array([0.01442363, 0.15595053, 0.27229018, ..., 0.70
038508, 0.11996656,
0.20053322], shape=(10000,))
```



Accessing elements

```
In [... arr = np.array([10, 20, 30, 40, 50])
```

```
arr[0] # first element
arr[-1] # last element
```

```
Out[... np.int64(10)
```

```
Out[... np.int64(50)
```

```
In [... # Slicing
arr[1:3]
arr[::2] # Every second element
arr[:2]
```

```
Out[... array([20, 30])
```

```
Out[... array([10, 30, 50])
```

```
Out[... array([10, 20])
```

```
In [... # Multi-dimensional arrays
arr2 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr2)
print(arr2[1, 2])
print(arr2[:, 2:]) # third column as a column vector
print(arr2[:, 2:].shape) # third column as a column v

[[1 2 3]
 [4 5 6]
 [7 8 9]]
6
[[3]
 [6]
 [9]]
(3, 1)
```

Boolean masking

Select elements using conditions.

```
In [... x = np.array([1, -2, 3, -4])
print(x[x > 0])

[1 3]
```

Aggregations with axis

Understand operations along dimensions.

```
In [... arr = np.array([[1,2,3],[4,5,6]])
print(arr)
arr.mean(axis=0)
arr.mean(axis=1)
```

```
[[1 2 3]
 [4 5 6]]
```

```
Out[... array([2.5, 3.5, 4.5])
```

```
Out[... array([2., 5.])
```

Views vs copies

Slicing usually returns a view.

```
In [... a = np.array([1,2,3,4])
# b=a.copy()
b = a[1:3].copy()
id(a)
id(b)

print(b)
b[0] = 99
b
a
```

```
Out[... 4878705232
```

```
Out[... 4878704464
```

```
[2 3]
```

```
Out[... array([99,  3])
```

```
Out[... array([1, 2, 3, 4])
```

```
In [... # a = [1,2,3,4]
# foo = []
# for i in a:
#     if i > 2:
#         # print(i)
#         foo.append(i)

# print(foo)

# numpy way
```

```
# print(list(range(5)))
a = np.array(range(5))
a
# b = a[:5] # b is a view
b = a[a>2] # b is a copy
b
b[0]=2
b
a
```

Out[...] array([0, 1, 2, 3, 4])

Out[...] array([3, 4])

Out[...] array([2, 4])

Out[...] array([0, 1, 2, 3, 4])

mathematical operations

In [...] `arr = np.array([0, np.pi / 2, np.pi])`

```
np.sin(arr)
np.cos(arr)
np.sqrt(arr)
import math
for i in arr:
    math.sqrt(i)
```

Out[...] array([0.00000000e+00, 1.00000000e+00, 1.2246468e-16])

Out[...] array([1.0000000e+00, 6.123234e-17, -1.0000000e+00])

Out[...] array([0. , 1.25331414, 1.77245385])

Out[...] 0.0

Out[...] 1.2533141373155001

Out[...] 1.7724538509055159

In [...] *# element-wise Operations*

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print("Addition:", arr1 + arr2)
print("Multiplication:", arr1 * arr2)
np.dot(arr1, arr2)
```

Addition: [5 7 9]
Multiplication: [4 10 18]

Out[... np.int64(32)

In [... *# Broadcasting: operations on arrays of different sha*

```
arr = np.array([1, 2, 3])  
print("Adding 5 to each element:", arr + 5)
```

Adding 5 to each element: [6 7 8]

Reshaping and Resizing

```
In [... arr = np.array([[1, 2, 3], [4, 5, 6]])  
arr  
# Reshape (2x3 to 3x2)  
reshaped_arr = arr.reshape((1,6))  
reshaped_arr  
  
arr.resize((3, 2)) # Resize (modifies the array in p  
arr
```

Out[... array([[1, 2, 3],
[4, 5, 6]])

Out[... array([1, 2, 3, 4, 5, 6])

Out[... array([[1, 2],
[3, 4],
[5, 6]])

Aggregation Functions: to compute summaries of arrays

```
In [... arr = np.array([10, 2, 3, 4, 5])  
arr.sum()  
arr.mean()  
arr.std()  
print(arr.min())  
arr.max()  
print(arr.argmax())
```

Out[... np.int64(24)

Out[... np.float64(4.8)

Out[... np.float64(2.785677655436824)


```
Out[... np.int64(10)
```

```
1
```

Vectorization vs loops

NumPy is designed to avoid explicit Python loops.

In the following example:

- `np.sin(x)` operates on the entire array at once
- This internal loop runs in compiled C, not in Python, avoiding Python loop overhead
- much faster execution for large arrays

```
In [... x = np.linspace(0, 2*np.pi, 50)
np.empty_like(x)
np.ones_like(x)
```

```
Out[... array([0.          , 0.12822827, 0.25645654, 0.3846848
1, 0.51291309,
          0.64114136, 0.76936963, 0.8975979 , 1.0258261
7, 1.15405444,
          1.28228272, 1.41051099, 1.53873926, 1.6669675
3, 1.7951958 ,
          1.92342407, 2.05165235, 2.17988062, 2.3081088
9, 2.43633716,
          2.56456543, 2.6927937 , 2.82102197, 2.9492502
5, 3.07747852,
          3.20570679, 3.33393506, 3.46216333, 3.5903916
, 3.71861988,
          3.84684815, 3.97507642, 4.10330469, 4.2315329
6, 4.35976123,
          4.48798951, 4.61621778, 4.74444605, 4.8726743
2, 5.00090259,
          5.12913086, 5.25735913, 5.38558741, 5.5138156
8, 5.64204395,
          5.77027222, 5.89850049, 6.02672876, 6.1549570
4, 6.28318531])
```

```
Out[... array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1
., 1., 1., 1., 1., 1.,
        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1
., 1., 1., 1., 1., 1.,
        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1
., 1., 1., 1., 1.]])
```

```
In [... x = np.linspace(0, 2*np.pi, 10000)

def loop_version(x):
    y = np.empty_like(x)
    for i in range(len(x)):
        y[i] = np.sin(x[i])
    return y

def vectorized_version(x):
    return np.sin(x)

# Run loop_version(x) `loop_num` times, measure the t

loop_num = 10

print('Loop version')
t_loop = timeit.timeit(lambda: loop_version(x), number=loop_num)
t_vector = timeit.timeit(lambda: vectorized_version(x), number=loop_num)
print(f'Loop version took {t_loop:.6f} seconds, while
```

Loop version

Loop version took 0.017060 seconds, while the vectorized version took 0.000555 seconds, making it about 30 times faster.